

LAB3-19 · PWM LED 밝기

전자전기컴퓨터설계실험Ⅱ · FPGA 응용 실습

버튼 한 번마다 duty를 10%씩 바꾸고 8개 LED에 같은 PWM을 출력한다.

Vivado 2026.1 · FPGA Part xc7s75fgga484-1

작성일 2026. 09. 21. · 이해리

목차

동작과 검증 목표

v2.0.2 프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code·Icarus 시뮬레이션

Vivado XSim·비트스트림

보드 실험과 레포트

이번 회로의 목표

버튼 한 번마다 duty를 10%씩 바꾸고 8개 LED에 같은 PWM을 출력한다.

핵심 개념: 50 MHz 단일 클록, 2단 동기화, 20 ms 디바운스, 한 클록 펄스, PWM duty

기능 TB: tb_led_pwm

예상 PASS: LAB3_LED_PWM_PASS checks=4

입출력과 보드 연결

clk B6 · reset K4 · 밝기 버튼 N8 · LED[7:0] N5/M1/M3/M7/N7/M2/M4/L4

모든 순차 로직은 MAIN CLOCK F의 50 MHz만 사용한다. 느린 동작은 내부 clock-enable로 만든다.

XDC의 포트 이름은 top module 선언과 글자·대소문자까지 같아야 한다.

TB가 자동으로 확인하는 범위

0%, 30%, 100%, 100% 다음 0% 순환에서 한 주기의 HIGH 클록 수를 검사한다.

시뮬레이션 파라미터는 실제 동작의 순서와 경계 조건을 유지하면서 시간을 줄인다.

PASS는 디지털 기능 검사 결과다. 핀 배선, 전기 규격, 실제 부품의 물리 동작은 보드 단계에서 확인한다.

v2.0.2 템플릿을 새 이름으로 clone

PowerShell에서 한 줄씩 실행한다.

```
1 git clone --branch v2.0.2 https://github.com/Glaysia/fpga-lab-template.git lab3_19_led_pwm
2 cd lab3_19_led_pwm
3 git switch -c main
4 code FPGA.code-workspace
```

태그 clone 직후 detached HEAD 안내는 정상이다. git switch -c main으로 자신의 이력을 시작한다.

빈 템플릿에서 만들 파일

경로	파일
src/	button_onepulse.v, lab3_led_pwm.v, pwm_channel.v
sim/	tb_led_pwm.sv
constraints/	lab3_led_pwm.xdc
프로젝트 루트	simulation.json

기존 빈 파일은 이름을 바꾸고, 필요한 추가 RTL은 src에 새로 만든다.

simulation.json 확인

sources에는 모든 RTL 파일을 의존 순서에 맞춰 넣는다.

testbench는 sim/tb_led_pwm.sv, simulation_top은 tb_led_pwm다.

설계 top lab3_led_pwm은 Vivado 합성에서 선택한다. TB module을 합성 top으로 선택하지 않는다.

코드를 읽고 입력하는 기준

다음 코드 화면은 줄 번호와 파일명이 원본 프로젝트와 연결되어 있다.

RTL은 합성 가능한 회로, TB는 입력 자극과 기대값 비교, XDC는 핀과 20 ns 클록 제약을 담당한다.

코드를 붙여 넣는 것으로 끝내지 말고 counter 폭, 파라미터 계산, 리셋 우선순위, 출력 시점을 설명할 수 있어야 한다.

simulation.json · 1-5행

```
1 {  
2   "sources": ["src/button_onepulse.v", "src/pwm_channel.v", "src/lab3_led_pwm.v"],  
3   "testbench": "sim/tb_led_pwm.sv",  
4   "simulation_top": "tb_led_pwm"  
5 }
```

button_onepulse.v · 1-18행

```
1 `timescale 1ns/1ps
2 module button_onepulse #(
3     parameter integer STABLE_CYCLES = 1_000_000
4 ) (
5     input wire clk,
6     input wire rst,
7     input wire button,
8     output reg pulse
9 );
10 (* ASYNC_REG = "TRUE" *) reg button_meta;
11 (* ASYNC_REG = "TRUE" *) reg button_sync;
12 localparam integer COUNT_WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
13 reg [COUNT_WIDTH-1:0] stable_count;
14 reg accepted;
15
16 always @(posedge clk or posedge rst) begin
17     if (rst) begin
18         button_meta <= 1'b0;
```

button_onepulse.v · 19-36행

```
19         button_sync <= 1'b0;
20     end else begin
21         button_meta <= button;
22         button_sync <= button_meta;
23     end
24 end
25
26 always @(posedge clk or posedge rst) begin
27     if (rst) begin
28         stable_count <= {COUNT_WIDTH{1'b0}};
29         accepted <= 1'b0;
30         pulse <= 1'b0;
31     end else begin
32         pulse <= 1'b0;
33         if (button_sync == accepted) begin
34             stable_count <= {COUNT_WIDTH{1'b0}};
35         end else if (stable_count == STABLE_CYCLES - 1) begin
36             stable_count <= {COUNT_WIDTH{1'b0}};
```

button_onepulse.v · 37–44행

```
37         accepted <= button_sync;  
38         pulse <= button_sync;  
39     end else begin  
40         stable_count <= stable_count + 1'b1;  
41     end  
42 end  
43 end  
44 endmodule
```

lab3_led_pwm.v · 1-18행

```
1 `timescale 1ns/1ps
2 module lab3_led_pwm #(
3     parameter integer CLK_HZ = 50_000_000,
4     parameter integer PWM_HZ = 1_000,
5     parameter integer LEVELS = 10,
6     parameter integer DEBOUNCE_CYCLES = 1_000_000
7 ) (
8     input wire clk,
9     input wire rst,
10    input wire button,
11    output wire [7:0] led
12 );
13    wire press;
14    wire pwm;
15    reg [3:0] level;
16
17    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button (
18        .clk(clk), .rst(rst), .button(button), .pulse(press)
```

lab3_led_pwm.v · 19-33행

```
19 );
20
21 always @(posedge clk or posedge rst) begin
22     if (rst)
23         level <= 4'd0;
24     else if (press)
25         level <= (level == LEVELS) ? 4'd0 : level + 1'b1;
26 end
27
28 pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm (
29     .clk(clk), .rst(rst), .level(level), .pwm(pwm)
30 );
31
32 assign led = {8{pwm}};
33 endmodule
```

pwm_channel.v · 1-18행

```
1 `timescale 1ns/1ps
2 module pwm_channel #(
3     parameter integer PERIOD_CYCLES = 50_000,
4     parameter integer LEVELS = 10
5 ) (
6     input wire clk,
7     input wire rst,
8     input wire [3:0] level,
9     output reg pwm
10 );
11     localparam integer COUNT_WIDTH = (PERIOD_CYCLES < 2) ? 1 : $clog2(PERIOD_CYCLES);
12     reg [COUNT_WIDTH-1:0] count;
13     integer threshold;
14
15     always @* begin
16         if (level >= LEVELS)
17             threshold = PERIOD_CYCLES;
18         else
```

pwm_channel.v · 19–34행

```
19         threshold = (PERIOD_CYCLES * level) / LEVELS;
20     end
21
22     always @(posedge clk or posedge rst) begin
23         if (rst) begin
24             count <= {COUNT_WIDTH{1'b0}};
25             pwm <= 1'b0;
26         end else begin
27             pwm <= (count < threshold);
28             if (count == PERIOD_CYCLES - 1)
29                 count <= {COUNT_WIDTH{1'b0}};
30             else
31                 count <= count + 1'b1;
32         end
33     end
34 endmodule
```

tb_led_pwm.sv · 1-18행

```
1 `timescale 1ns/1ps
2 module tb_led_pwm;
3     reg clk=0, rst=1, button=0;
4     wire [7:0] led;
5     integer checks=0, high_count, i;
6     always #5 clk=~clk;
7     lab3_led_pwm #(.CLK_HZ(1000),.PWM_HZ(100),.LEVELS(10),.DEBOUNCE_CYCLES(2)) dut
8         (.clk(clk),.rst(rst),.button(button),.led(led));
9
10    task press_button;
11    begin
12        button=1; repeat(6) @(posedge clk);
13        button=0; repeat(6) @(posedge clk);
14    end endtask
15
16    task expect_level(input integer expected);
17    begin
18        wait(dut.u_pwm.count==0); high_count=0;
```

tb_led_pwm.sv · 19-34행

```
19     for(i=0;i<10;i=i+1) begin @(posedge clk); #1; if(led[0]) high_count=high_count+1; end
20     if(high_count!=expected) $fatal(1,"level=%0d high=%0d",expected,high_count);
21     checks=checks+1;
22 end endtask
23
24 initial begin
25     $dumpfile("wave.vcd"); $dumpvars(0,tb_led_pwm);
26     repeat(3) @(posedge clk); rst=0;
27     expect_level(0);
28     repeat(3) press_button(); expect_level(3);
29     repeat(7) press_button(); expect_level(10);
30     press_button(); expect_level(0);
31     $display("LAB3_LED_PWM_PASS checks=%0d",checks); $finish;
32 end
33 initial begin #20000; $fatal(1,"timeout"); end
34 endmodule
```

lab3_led_pwm.xdc · 1-15행

```
1 # Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.  
2 set_property PACKAGE_PIN B6 [get_ports clk]  
3 set_property PACKAGE_PIN K4 [get_ports rst]  
4 set_property PACKAGE_PIN N8 [get_ports button]  
5 set_property PACKAGE_PIN N5 [get_ports {led[0]}]  
6 set_property PACKAGE_PIN M1 [get_ports {led[1]}]  
7 set_property PACKAGE_PIN M3 [get_ports {led[2]}]  
8 set_property PACKAGE_PIN M7 [get_ports {led[3]}]  
9 set_property PACKAGE_PIN N7 [get_ports {led[4]}]  
10 set_property PACKAGE_PIN M2 [get_ports {led[5]}]  
11 set_property PACKAGE_PIN M4 [get_ports {led[6]}]  
12 set_property PACKAGE_PIN L4 [get_ports {led[7]}]  
13 set_property IOSTANDARD LVCMOS33 [get_ports *]  
14 create_clock -name main_50mhz -period 20.000 [get_ports clk]  
15 set_false_path -from [get_ports {rst button}]
```

VS Code에서 Icarus 실행

1. File → Save All.
2. Terminal → Run Task... → 01 Check tools.
3. Git·Python·iverilog·vvp 경로와 버전을 확인한다.
4. Terminal → Run Task... → 02 Simulate.
5. LAB3_LED_PWM_PASS checks=4와 종료 시각을 기록한다.
6. 03 Open waveform으로 build/sim/wave.vcd를 연다.

VS Code · 프로젝트와 파일

실제 화면 캡처 자리

ID: 01_led_pwm-vscode-workspace

FPGA.code-workspace를 연 뒤 Explorer에 src.sim.constraints.tools와 작성 파일이 보이게 캡처한다.

VS Code · 시뮬레이션 PASS

실제 화면 캡처 자리

ID: 01_led_pwm-vscode-pass

02 Simulate 실행 뒤 LAB3_LED_PWM_PASS checks=4와 종료 시각이
함께 보이게 캡처한다.

VS Code · 파형

실제 화면 캡처 자리

ID: 01_led_pwm-vscode-wave

TB top을 펼쳐 clk·rst·주요 입력·내부 상태·출력을 추가하고
경계 조건이 보이게 확대한다.

파형에서 설명할 내용

0%, 30%, 100%, 100% 다음 0% 순환에서 한 주기의 HIGH 클록 수를 검사한다.

입력을 바꾼 시점, 유효 상승 에지, 상태 갱신 직후를 구분한다.

파라미터를 줄인 TB 시간과 실제 50 MHz 보드 시간을 혼동하지 않는다.

수정 실험

LEVELS를 10에서 5로 바꾸어 단계 수와 파형의 HIGH 폭을 비교한 뒤 복구한다.

정상 PASS와 파형을 먼저 보관한 뒤 한 항목만 수정한다. 예상값, 실제 FAIL 또는 변화, 복구 PASS를 3행 표로 기록한다.

TB의 기대값을 RTL 오류에 맞춰 바꾸지 않는다.

Vivado 2026.1 프로젝트 생성

1. Create Project → RTL Project → Do not specify sources at this time.
2. Part에 xc7s75fgga484-1을 정확히 선택한다.
3. Add Sources에 src의 RTL 전부, Add Simulation Sources에 TB를 추가한다.
4. Add or Create Constraints에 XDC를 추가한다.
5. Design Sources의 top을 lab3_led_pwm으로 설정한다.

Vivado · 소스와 top

실제 화면 캡처 자리

ID: 01_led_pwm-vivado-sources

Sources 창에서 RTL, Simulation Sources의 TB, Constraints의 XDC
와 top module이 함께 보이게 캡처한다.

Vivado XSim

Flow Navigator → Simulation → Run Behavioral Simulation.

Tcl Console의 LAB3_LED_PWM_PASS checks=4를 확인한다. Objects/Wave에서 입력과 출력을 추가하고 Zoom Fit으로 전체 실행을 본다.

Icarus와 XSim은 같은 RTL·TB를 실행해야 한다. 서로 다른 사본을 등록하지 않는다.

Vivado · XSim PASS와 파형

실제 화면 캡처 자리

ID: 01_led_pwm-xsim-pass

Tcl Console PASS, waveform 이름, 시간축, 핵심 입출력이 한 화면에 보이게 캡처한다.

핀과 클록 제약 확인

Open Elaborated Design 또는 I/O Planning에서 top port와 PACKAGE_PIN을 대조한다.

Reports → Timing → Report Clocks에서 clk 주기 20.000 ns를 확인한다.

XDC를 저장한 뒤 Run Synthesis → Run Implementation → Generate Bitstream 순서로 실행한다.

Vivado · 구현과 bit 생성

실제 화면 캡처 자리

ID: 01_led_pwm-vivado-bit

Implementation 완료, Timing Summary의 WNS, DRC 오류 0건,
bitstream 생성 완료가 확인되는 화면을 캡처한다.

보드에서 확인

전원을 끈 상태에서 외부 부품 결선과 공통 GND를 먼저 확인한다.

Hardware Manager → Open Target → Auto Connect → Program Device에서
생성한 bit를 기록한다.

버튼을 한 번씩 눌러 단계가 하나씩 이동하는지, 0%와 100% 및 중간 밝기를
관찰한다.

사진에는 보드 전체와 사용 핀이, 영상에는 입력 조작과 출력 변화가 함께
보이게 한다.

실험 전 레포트

1. 동작 원리와 핵심 파라미터 계산을 자기 말로 설명한다.
2. RTL·TB·XDC 각 파일의 역할과 top module을 적는다.
3. TB 자극과 기대 결과를 표로 정리한다.
4. Icarus PASS·파형과 수정/복구 결과를 넣고 경계 조건을 설명한다.
5. 보드에서 확인해야 할 항목과 예상 결과를 적는다.

실험 후 레포트와 제출

1. XSim PASS·파형, DRC·timing·bitstream 결과를 기록한다.
2. 실제 보드 관찰, 계산값과 실측의 차이, 원인을 설명한다.
3. 결선이 보이는 사진과 동작 영상 링크를 첨부한다.
4. 자신의 GitHub 저장소 URL과 최종 commit hash를 적는다.
5. src·sim·constraints·simulation.json만 최종 원본으로 관리하고 생성물은 제외한다.

완료 체크

- ☐ v2.0.2 태그에서 새 프로젝트를 만들었다.
- ☐ RTL·TB·XDC를 직접 작성하고 설명할 수 있다.
- ☐ Icarus와 XSim에서 같은 PASS를 확인했다.
- ☐ 핀·20 ns 클록·DRC·timing을 확인하고 bit를 만들었다.
- ☐ 보드 동작 사진·영상과 GitHub 링크를 정리했다.
- ☐ 실험 전·후 레포트에 결과와 한계를 구분해 기록했다.